# Roteiro completo da apresentação

Padrão REST & Boas Práticas na Criação de APIs | Pabllo (vídeo gravado) e Gabriel Foletto (ao vivo)

| Parte | Quem | Conteúdo | Tempo estimado |
|---|---|---|---|
| 1 | Pabllo (vídeo) | Abertura, Roy Fielding, 6 pilares, verbos HTTP, idempotência | cerca de 7 min |
| 2 | Gabriel (ao vivo) | Status codes, URIs, paginação e filtros, versionamento | cerca de 6 min |
| 3 | Gabriel (ao vivo) | O projeto, demonstração no Swagger e no Postman | cerca de 8 min |
| 4 | Gabriel (ao vivo) | Conclusão | cerca de 1 min |

**Aviso sobre o tempo:** o tempo oficial do seminário ainda não foi informado, então estimei cerca de 21 minutos no total. Se o professor der menos, dá para cortar: o item 'se der tempo' da demonstração no Swagger e a explicação detalhada dos três tipos de versionamento. Se der mais, mostre mais cenários no Postman.

**Como ler este roteiro:** o texto em letra grande é o que se fala. As caixas coloridas **NA TELA** dizem o que mostrar enquanto fala. A parte 1 é para o vídeo do Pabllo, e as partes 2, 3 e 4 são do Gabriel.

## PARTE 1 | PABLLO (vídeo) | cerca de 7 min

### 1.1 Abertura

*0:00 a 0:45*

Olá, pessoal! Eu sou o Pabllo e, junto com o Gabriel Foletto, vou apresentar o tema **Padrão REST e Boas Práticas na Criação de APIs**.

Como eu não vou estar presente na sala, essa minha parte está gravada, e o Gabriel continua a apresentação ao vivo.

A divisão é simples. Eu explico a **base teórica**: de onde o REST surgiu, quais são os seus 6 pilares e como usar os verbos HTTP, incluindo a idempotência. Depois, o Gabriel apresenta as boas práticas e o **sistema funcionando**. Vamos começar.

### 1.2 O que é REST e quem criou

*0:45 a 1:45*

Primeiro, o que é REST? É a sigla de **Representational State Transfer**, em português, Transferência de Estado Representacional.

Um ponto importante: REST **não é uma tecnologia, nem um framework, nem um protocolo**. É um **estilo de arquitetura**, ou seja, um conjunto de regras e restrições para construir sistemas na web.

Ele foi definido por **Roy Fielding**, na tese de doutorado dele, no ano 2000, na Universidade da Califórnia, em Irvine. O Fielding não era um teórico distante: ele foi um dos autores da especificação do HTTP e um dos criadores do servidor Apache. Na tese, ele descreveu os princípios que fizeram a própria web crescer em escala.

Quando uma API segue essas regras, a gente chama de uma API **RESTful**.

### 1.3 Os 6 pilares arquiteturais

*1:45 a 4:45*

Agora, os 6 pilares. Vou explicar um por um.

**Primeiro: cliente-servidor.** Cliente e servidor têm responsabilidades separadas. O cliente cuida da interface, do que o usuário vê. O servidor cuida dos dados e das regras de negócio. Assim, os dois evoluem de forma independente, e a mesma API pode atender um site, um aplicativo de celular e outros sistemas ao mesmo tempo.

**Segundo: stateless, ou sem estado.** Cada requisição precisa levar tudo o que o servidor precisa para entendê-la. O servidor não guarda a sessão do cliente entre uma chamada e outra. Por exemplo, o cliente envia o token de autenticação em toda requisição. A vantagem é a escalabilidade: qualquer servidor pode responder qualquer requisição.

**Terceiro: cache.** As respostas devem informar se podem ser armazenadas, e por quanto tempo, com cabeçalhos como Cache-Control e ETag. O resultado é menos carga no servidor e respostas mais rápidas.

**Quarto: interface uniforme.** Esse é o pilar mais característico do REST. Toda a comunicação segue um padrão único. Os recursos são identificados por URIs. Eles são manipulados por representações, normalmente em JSON. As mensagens são autodescritivas: o verbo, o status e os cabeçalhos já dizem o que está acontecendo. E existe o conceito de **HATEOAS**, em que a resposta traz links com as próximas ações possíveis.

**Quinto: sistema em camadas.** O cliente não precisa saber se está falando com o servidor final ou com algo no meio do caminho, como um balanceador de carga, um gateway ou um proxy. Isso ajuda na segurança e na escalabilidade.

**Sexto e último: código sob demanda.** É o único pilar **opcional**. O servidor pode enviar código executável para o cliente, como um JavaScript. Na prática, é raro em APIs, mas faz parte da definição original.

### 1.4 Verbos HTTP

*4:45 a 5:45*

Agora, os verbos HTTP. No REST, a regra é: a **URI identifica o recurso**, que é um substantivo, por exemplo, barra produtos. E o **verbo diz a ação** que queremos fazer nele.

O **GET** serve para ler, para consultar. Ele não altera nada no servidor. O **POST** serve para criar um novo recurso. O **PUT** serve para atualizar, substituindo o recurso inteiro: você manda todos os campos. O **PATCH** serve para atualizar só uma parte: você manda apenas o campo que mudou. E o **DELETE** serve para remover o recurso.

### 1.5 Idempotência

*5:45 a 6:35*

Entra aqui um conceito muito cobrado: a **idempotência**. Uma operação é idempotente quando, ao executá-la uma vez ou várias vezes com a mesma requisição, o **estado final no servidor é o mesmo**.

Pense no botão do elevador: apertar uma vez ou dez vezes dá no mesmo. Na API, o **PUT** é idempotente: se eu mandar dez vezes 'o nome é Ana', no final o nome continua sendo Ana. O **DELETE** também: apagar o cliente uma vez ou cinco vezes deixa o sistema igual, porque o cliente já não existe. O **GET** também é idempotente, e ainda é seguro, porque não altera nada.

Já o **POST não é idempotente**. Se eu enviar a mesma requisição três vezes, posso criar três pedidos. Por isso, em um pagamento, um clique duplo pode gerar uma cobrança duplicada. E o **PATCH depende** da operação: 'definir o status como pago' é

idempotente, mas 'somar 1 ao estoque' não é.

### 1.6 Fechamento e passagem para o Gabriel

*6:35 a 7:15*

Resumindo: o REST é um estilo de arquitetura criado por Roy Fielding, baseado em 6 pilares. Os verbos dizem a ação, e a idempotência garante previsibilidade quando uma requisição é repetida.

Mas a teoria só faz sentido na prática. Agora eu passo a palavra para o **Gabriel**, que vai apresentar as boas práticas e o nosso sistema funcionando. Gabriel, é com você! Obrigado, pessoal.

## PARTE 2 | GABRIEL (ao vivo) | Boas práticas | cerca de 6 min

### 2.1 Retomada

*0:00 a 0:30*

Obrigado, Pabllo! Boa noite, professor e pessoal. Eu sou o Gabriel Foletto. O Pabllo explicou a base: o que é REST, os 6 pilares, os verbos HTTP e a idempotência. Agora eu vou mostrar como isso vira prática. Primeiro, quatro boas práticas de uma API: os **códigos de status**, a **estrutura das URIs**, a **paginação com filtros** e o **versionamento**. Depois, eu apresento a API que desenvolvemos, funcionando.

### 2.2 Códigos de status

*0:30 a 2:30*

Começando pelos códigos de status. Eles são a forma de a API dizer o que aconteceu, sem o cliente precisar ler a mensagem. Eles são divididos em famílias.

A família **2xx** significa que deu certo. O **200 OK** diz que a requisição funcionou e vem o resultado. Usamos no GET e no PUT. O **201 Created** diz que algo foi criado. Usamos no POST, junto com o cabeçalho Location, que aponta o endereço do novo recurso. E o **204 No Content** diz que deu certo, mas não há nada para devolver. Usamos no DELETE.

A família **4xx** é erro do cliente, ou seja, quem chamou a API errou. O **400 Bad Request** é para dados inválidos, como um preço negativo. O **404 Not Found** é quando o recurso não existe. O **405 Method Not Allowed** é quando o verbo não é permitido naquela rota. E o **409 Conflict** é um conflito, como cadastrar um produto com um nome que já existe. Existem também o 401 e o 403, para autenticação e permissão, que não implementamos neste projeto.

A família **5xx** é erro do servidor. O **500 Internal Server Error** significa que algo inesperado deu errado do nosso lado. Nesse caso, nunca devolvemos detalhes internos ao cliente.

A regra de ouro: **2xx é sucesso, 4xx é culpa de quem chamou, 5xx é culpa de quem atende**. Um erro comum é devolver 200 com uma mensagem de erro dentro. Isso quebra o sentido do protocolo.

> **NA TELA (slide)**
> - Tabela: 200, 201, 204 (2xx) | 400, 404, 405, 409 (4xx) | 500 (5xx)
> - Frase de destaque: 2xx sucesso | 4xx erro do cliente | 5xx erro do servidor

| Código | Nome | Quando usar (no projeto) |
|---|---|---|
| 200 | OK | GET e PUT com sucesso |
| 201 | Created | POST criou o produto (header Location) |

| Código | Nome | Quando usar (no projeto) |
|---|---|---|
| 204 | No Content | DELETE com sucesso |
| 400 | Bad Request | Dados inválidos, JSON quebrado, id inválido |
| 404 | Not Found | Produto ou rota não existe |
| 405 | Method Not Allowed | Verbo não permitido na rota |
| 409 | Conflict | Nome de produto já existe |
| 500 | Internal Server Error | Erro inesperado no servidor |

## 2.3 Estrutura das URIs

*2:30 a 4:00*

Agora, a estrutura das URIs. A URI identifica o recurso, então usamos **substantivos, no plural, e nunca verbos**. O verbo quem dá é o HTTP.

Um exemplo ruim é 'barra buscarProdutos' ou 'barra criarProduto'. O exemplo bom é 'GET barra produtos' para listar e 'POST barra produtos' para criar. Para um produto específico: 'barra produtos barra 1'. Se existir relação entre recursos, a URI segue a hierarquia, por exemplo 'barra clientes barra 1 barra pedidos'.

Outras boas práticas: letras minúsculas, hífen para separar palavras, sem barra no final, e os **filtros na query string**, depois do ponto de interrogação, nunca no meio do caminho.

> **NA TELA (slide)**
>
> - Ruim: /buscarProdutos | /criarProduto | /produto/deletar/1
> - Bom: GET /produtos | POST /produtos | DELETE /produtos/1
> - Relação: /clientes/1/pedidos
> - Filtro: /produtos?nome=note (query string)

## 2.4 Paginação e filtros

*4:00 a 5:30*

Imagine uma API com um milhão de produtos. Devolver tudo seria lento e pesado. Por isso usamos **paginação**: o cliente pede uma página por vez, com os parâmetros **page** e **size**. No nosso projeto, 'produtos, interrogação, page igual a 0, size igual a 5'.

A resposta vem com a lista dentro de **content**, e com as informações para navegar: a página atual, o tamanho, o total de elementos e o total de páginas. Também colocamos um limite: o size vai de 1 a 100, e fora disso a API devolve 400.

Os **filtros** usam a mesma ideia, com parâmetros na query string. No projeto temos **nome**, **precoMin** e **precoMax**. Filtros e paginação podem ser combinados na mesma chamada.

**NA TELA (slide)**

- GET /v1/produtos?page=0&size=5
- GET /v1/produtos?nome=note&precoMin=1000
- Resposta: content | page | size | totalElements | totalPages

## 2.5 Versionamento

*5:30 a 6:30*

Por último, o **versionamento**. APIs evoluem, e se mudarmos o formato de uma resposta, quem já usa pode quebrar. O versionamento permite lançar uma versão nova sem derrubar a antiga.

Existem três formas comuns: **na URL**, como 'barra v1 barra produtos'; **em um cabeçalho HTTP**; ou **em um parâmetro de consulta**. A mais usada e mais simples de entender é na URL, e foi a que usamos: a nossa API está em **barra v1**. Quando houver uma mudança que quebra compatibilidade, criamos a v2 e mantemos a v1 funcionando por um tempo.

**NA TELA (slide)**

- Na URL: /v1/produtos (a que usamos)
- No cabeçalho: Accept-Version: 1
- Na query string: /produtos?version=1
- Mudou de forma incompatível: cria /v2 e mantém a /v1

## PARTE 3 | GABRIEL (ao vivo) | O projeto e a demonstração | cerca de 8 min

**ANTES DE COMEÇAR (checklist)**

- Reinicie a API para os dados voltarem ao início (Notebook, Iphone e Mouse).
- Deixe a API rodando, e o Swagger aberto em http://localhost:8080/swagger-ui.html em uma aba.
- Deixe o Postman aberto com a coleção importada e o ambiente 'Variaveis Api Pedidos' selecionado.
- Deixe o projeto aberto na IDE, com a árvore de pastas visível.
- Plano B: se a API não subir, mostre o Swagger e o Postman em prints, e explique os cenários.

### 3.1 Apresentando o projeto

*0:00 a 1:15*

Vamos ao projeto. Desenvolvemos uma **API RESTful de gerenciamento de produtos**, em Java 17 com Spring Boot. Usamos Maven, Lombok, o Spring Validation para validar os dados e o Springdoc para gerar o Swagger. Os dados ficam em memória, porque o foco aqui são os conceitos de REST.

O código está organizado em **camadas**. O **controller** recebe as requisições e define as rotas. O **service** guarda as regras de negócio. O **dto** representa os dados que entram e faz a validação. O **model** é a entidade Produto, com id, nome, preço e quantidade. E o **handler** trata os erros em um só lugar, transformando cada exceção no status correto e em um JSON padronizado.

A API implementa o **CRUD completo**: GET para listar e buscar por id, POST para criar, PUT para substituir, PATCH para atualizar parcialmente e DELETE para remover.

**NA TELA**

- Mostrar a árvore de pastas: controller, service, dto, database/model, exeption, handler.
- Se possível, abrir o ProdutoController e apontar os verbos (@GetMapping, @PostMapping, @PutMapping, @PatchMapping, @DeleteMapping).

### 3.2 Demonstração no Swagger

*1:15 a 4:00*

Agora, o **Swagger**. Ele é a documentação interativa da API, gerada automaticamente a partir do código, no padrão **OpenAPI**. Aqui vemos todos os endpoints, os parâmetros e os status possíveis de cada um. E eu posso testar direto daqui.

**NA TELA | passo a passo (essencial)**

- **1. GET /v1/produtos**: clique em Try it out > Execute. Fale: 'Devolveu 200, com a lista paginada: content, page, size, totalElements e totalPages.'
- **2. Mesmo GET com size = 2**: Execute. Fale: 'Agora são 2 por página, e o totalPages mostra que existem mais páginas.'
- **3. POST /v1/produtos** com o corpo {"nome":"Monitor","preco":900,"quantidade":8}: Execute. Fale: '201 Created. Olhem o cabeçalho Location, que aponta para o novo produto.'
- **4. Executar o mesmo POST de novo**: Fale: '409 Conflict, porque o nome já existe. O POST não é idempotente.'
- **5. POST com dados inválidos** {"nome":"","preco":-5,"quantidade":1}: Fale: '400 Bad Request, e a mensagem diz exatamente quais campos estão errados.'
- **6. GET /v1/produtos/99**: Fale: '404 Not Found, com a mensagem padronizada de erro.'

**NA TELA | se der tempo**

- **PUT /v1/produtos/1** com {"nome":"Notebook Gamer","preco":6000,"quantidade":5}: '200 OK, substitui o produto inteiro.'
- **PATCH /v1/produtos/1** com {"quantidade":20}: '200 OK, só a quantidade mudou.'
- **DELETE /v1/produtos/3**: '204 No Content.' Executar de novo: '204 de novo. É idempotente: o estado final é o mesmo.'

### 3.3 Demonstração no Postman

*4:00 a 7:00*

Agora, o **Postman**. Montamos uma coleção com **22 requisições**, organizadas em 6 pastas: leitura, filtros e paginação, criação, atualização, remoção e erros de protocolo. Cada requisição tem um **teste automático** que confere se a API devolveu o status esperado.

Vou rodar a coleção inteira com o **Runner**. Cada teste verde significa que a API respondeu exatamente o que esperávamos.

**NA TELA | passo a passo**

- Mostre a barra lateral com as 6 pastas e abra uma requisição (por exemplo, 'Criar válido') para mostrar o corpo e a URL com a variável {{baseUrlPedidos}}.
- Nos três pontinhos da coleção: **Run** > botão **Run**.
- Quando terminar, aponte os resultados e comente os grupos abaixo.

Reparem que temos cenários de **sucesso**, com 200, 201 e 204, e de **erro**, com 400, 404, 405 e 409. Olhem o **PUT repetido** e o **DELETE repetido**: os dois dão o mesmo resultado, que é a idempotência na prática. Já o **POST repetido** dá 409, porque ele não é idempotente. Também temos o erro 405, quando usamos um verbo que a rota não aceita, e o 404, quando a rota nem existe.

Resumindo a demonstração: a mesma API foi testada de duas formas. O **Swagger** serve para documentar e explorar. O **Postman** serve para testar de forma repetível,

com cenários de sucesso e de erro.

## PARTE 4 | GABRIEL (ao vivo) | Conclusão | cerca de 1 min

Para concluir. Vimos que o **REST** é um estilo de arquitetura criado por Roy Fielding, baseado em **6 pilares**. Vimos como usar os **verbos HTTP** e a **idempotência**, como devolver **status codes** corretos, como estruturar **URIs**, aplicar **paginação e filtros** e fazer o **versionamento**. E vimos tudo isso funcionando em uma API de produtos, documentada no **Swagger** e testada no **Postman**.

Seguir essas boas práticas deixa a API mais previsível, mais fácil de usar e de manter. Muito obrigado, professor e pessoal. Ficamos à disposição para perguntas.

## Sugestão de slides do Gabriel

| # | Slide | Conteúdo |
|---|---|---|
| 1 | Retomada | Título do tema + 'O que vamos ver': status, URIs, paginação/filtros, versionamento, projeto |
| 2 | Códigos de status | Tabela 2xx / 4xx / 5xx |
| 3 | URIs | Exemplos ruins x bons |
| 4 | Paginação e filtros | URLs de exemplo e formato da resposta (content, page, size, totalElements, totalPages) |
| 5 | Versionamento | 3 formas e a que usamos (/v1) |
| 6 | O projeto | Tecnologias e camadas (controller, service, dto, model, handler) |
| 7 | Endpoints | Tabela de verbos, rotas e status |
| 8 | Demonstração | Só uma tela de transição: 'Swagger e Postman ao vivo' |
| 9 | Conclusão | Resumo em 5 tópicos e 'Obrigado' |